Failure (force the parser to actually fail) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `structured_output_practice.py` (Failure section): you need to know, before it happens in production, exactly which exception a bad reply raises. "Shape during" almost never fails, so this exercise uses "check after" — a `PydanticOutputParser` — where a reply in the wrong shape really does fail. **If not:** the Build Task's "a reply that doesn't fit the shape raises a clear, named error" test would check an error you had never seen fire.

Basic Version

Approach 1 — the direct way

```
# structured_output_practice.py — Failure section
from pydantic import BaseModel
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI


class Rating(BaseModel):
    score: int


# how: "check after" — the prompt never tells the model the shape,
# so the parser only finds out after the reply is written
parser = PydanticOutputParser(pydantic_object=Rating)
prompt = ChatPromptTemplate.from_template("Rate {thing} from 1 to 10.")
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
chain = prompt | model | parser

try:
    chain.invoke({"thing": "pizza"})
except Exception as e:
    print("Error type:", type(e).__name__)
```

Expected output:

```
Error type: OutputParserException
```

The prompt never mentioned a shape, so the model answered in a sentence ("I'd rate pizza a 9 out of 10!"), and the parser couldn't read it as JSON. Catching the broad `Exception` finds the class — Intermediate catches it by name.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — catch it by name, then fix the prompt

```
# structured_output_practice.py — Failure section
from pydantic import BaseModel
from langchain_core.exceptions import OutputParserException
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI


class Rating(BaseModel):
    score: int


parser = PydanticOutputParser(pydantic_object=Rating)

# why: no word about the shape — the model answers in a sentence
BROKEN_TEMPLATE = "Rate {thing} from 1 to 10."
# how: {format_instructions} is filled once with .partial(), so the
# caller only passes {thing}
FIXED_TEMPLATE = "Rate {thing} from 1 to 10.\n{format_instructions}"


def build_rating_chain(template: str) -> Runnable:
    prompt = ChatPromptTemplate.from_template(template)
    if "{format_instructions}" in template:
        instructions = parser.get_format_instructions()
        prompt = prompt.partial(format_instructions=instructions)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return prompt | model | parser


def get_rating(chain: Runnable, thing: str) -> Rating | None:
    try:
        return chain.invoke({"thing": thing})
    except OutputParserException:
        # why: catch the parser's own error by name — a network error
        # or a bad key must still fail loudly
```

```
        return None


def main() -> None:
    broken_chain = build_rating_chain(BROKEN_TEMPLATE)
    print(f"case 1 (no instructions)   -> {get_rating(broken_chain, 'pizza')}")

    fixed_chain = build_rating_chain(FIXED_TEMPLATE)
    print(f"case 2 (with instructions) -> {get_rating(fixed_chain, 'pizza')}")

    # how: .parse() alone checks any text — no API call needed
    try:
        parser.parse("I'd say eight")
        print("case 3 (parse by hand)     -> accepted (wrong!)")
    except OutputParserException:
        print("case 3 (parse by hand)     -> OutputParserException")


if __name__ == "__main__":
    main()
```

Expected output (the score varies):

```
case 1 (no instructions)   -> None
case 2 (with instructions) -> score=9
case 3 (parse by hand)     -> OutputParserException
```

Difference from Basic: `OutputParserException` is caught by name inside `get_rating()`, which returns `None` — anything else still fails loudly. The fixed template adds `parser.get_format_instructions()` with `.partial()`, so the model knows the shape. And `parser.parse()` tests the failure with no API call.

Which one should you write? Intermediate's. The Build Task's test file uses the same `parser.parse(...)` check for its "wrong shape" case.